实验二 启发式搜索与博弈搜索实验报告

姓名：李俐彤　学号：10245501415

1 实验目的

1. 掌握A*启发式搜索算法的基本原理，分别实现普通网格迷宫最短路径搜索、加权地形最小代价搜索，理解评估函数 $((f(n)) = (g(n)) + (h(n)))$ 的含义。
2. 理解**可采纳启发函数**的定义，对比曼哈顿距离在无权迷宫、加权地形两种场景下的性质差异，认识启发函数对搜索效率和最优性的影响。
3. 掌握极大极小（Minimax）博弈搜索思想，实现Alpha-Beta剪枝算法，理解剪枝条件，体会剪枝对博弈树搜索节点数量的优化作用。
4. 设计多组测试用例验证算法正确性，分析算法特点、边界情况与局限性。

2 实验环境

编程语言：Python 3.7+
依赖库：heapq（优先队列）、typing（类型注解），均为Python内置库，无需额外安装第三方包
运行环境：Windows，VS Code

3 算法原理与程序设计

3.1 A*启发式搜索

A*算法依靠评估函数：

$$((f(n)) = (g(n)) + (h(n)))$$

- $(g(n))$：从起点到达节点 n 的**实际累计代价**，是已经确定的开销；
- $(h(n))$：启发函数，对节点 n 到终点的剩余代价进行估计；
- $(f(n))$：节点综合优先级，算法每次从Open优先队列中选取 $(f(n))$ 最小的节点进行扩展。
  两个核心集合：
- Open集合：优先队列，存放待扩展节点；
- 代价记录表+父节点字典：记录到达每个节点的最小代价，并且用于在搜到终点时回溯生成完整路径。
  本次实验选用**曼哈顿距离**作为启发函数：$h = |x_1 - x_2| + |y_1 - y_2|$，仅允许上下左右4方向移动，遇到障碍物X则跳过。
- 无权迷宫：每一步代价固定为1，曼哈顿距离满足 $h(n) \le h^*(n)$，属于可采纳启发，A一定可以找到全局最短路径；
- 加权地形：不同格子移动代价不同，曼哈顿距离不再满足可采纳条件，算法仍可运行，但启发函数不再保证下界约束。

核心伪代码

```
初始化优先队列Open，起点入队；g记录代价，parent记录父节点
while Open不为空：
    取出f值最小的节点u
    if u是终点：回溯父节点，输出路径
    遍历四个方向邻居v
        计算新代价new_g
        if new_g < 已知代价：更新代价、父节点，压入优先队列
无解，返回空列表 / -1
```

3.2 Alpha-Beta剪枝博弈搜索

博弈树为双人零和博弈，两个玩家轮流行动：

- MAX玩家（根节点）：选择子节点，最大化最终评估得分；
- MIN玩家：选择子节点，最小化最终评估得分；
- 叶子节点：直接返回博弈局面评估值。
  参数定义：
- α：MAX玩家当前能够保证拿到的分数**下界**（最大值）；
- β：MIN玩家当前能够保证拿到的分数**上界**（最小值）； 剪枝条件：$\alpha \geq \beta$，满足时停止遍历该节点剩余子节点，剪掉不必要分支，减少搜索量。
  重要：子节点必须从左向右依次遍历，访问顺序直接决定剪枝效果。
  核心伪代码

```
function alpha_beta(node,α,β, is_max):
    if 是叶子节点：return node评估值
    if 当前是MAX节点：
        val = -∞
        for child in children：
            val = max(val, alpha_beta(child,α,β,False))
            α= max(α, val)
            if α>=β：
            break（β剪枝）
        return val
    else：# MIN节点
        val = +∞
        for child in children：
            val = min(val, alpha_beta(child,α,β,True))
            β= min(β, val)
            if α >= β：break（α剪枝）
        return val
```

3.3 宝藏争夺综合问题原理

该问题将 A * 加权路径搜索与深度受限对抗搜索结合。 收益计算公式：
$Gain(p, T_i) = V_i - d(p, T_i)$
V_i为宝藏价值，$d(p, T_i)$为玩家当前位置p到宝藏T_i的最小移动代价。
局面评估函数：$\boldsymbol{U = Score_A - Score_B}$。

- A 为 MAX 玩家，目标最大化U；B 为 MIN 玩家，目标最小化U。
- 搜索深度限制H：控制博弈向前预测的回合数；到达深度上限或者宝藏全部取完则直接返回评估值U。

> 为降低计算开销，预先使用 A * 计算所有关键点两两之间最短移动代价，博弈递归时直接查表。
> 博弈状态包含：A 坐标、B 坐标、剩余宝藏集合、A 累计得分、B 累计得分、当前搜索深度、行动玩家。

4 测试用例与实验结果

```
[Running] python -u "c:\Users\lilitong\Desktop\d大三上\当代人工智能
\实验二\search_exp2.py"
===== Test4: Treasure Competition =====
===== Test1: Maze A* =====
Path: [(0, 0), (0, 1), (0, 2), (1, 2), (1, 3), (1, 4), (2, 4),
(3, 4)]
Total cost: 7

===== Test2: Weighted terrain A* =====
Min total cost = 7

===== Test3: Game Tree Alpha-Beta =====
MAX best score = 5

H=2: {'best_target':'None', 'best_value':-inf}
H=4: {'best_target':'None', 'best_value':-inf}


[Done] exited with code=0 in 0.516 seconds
```

4.1 测试1：无权迷宫A*寻路

迷宫网格：

```
['S', '.', '.', 'X', '.'],

['.', 'X', '.', '.', '.'],

['.', 'X', '.', 'X', '.'],

['.', '.', '.', 'X', 'E']
```

起点(0,0)，终点(3,4)，X代表障碍物。
输出路径： [(0, 0), (0, 1), (0, 2), (1, 2), (1, 3), (1, 4), (2, 4), (3, 4)]
结果分析：路径避开全部障碍物，是网格中的最短路径，算法验证正确。

4.2 测试2：加权地形A*最小代价搜索

输入加权地图规则：移动到某个格子，代价等于该格子数字；起点 S、终点 E 不计代价。

```
['S', '1', '1', '8', '1'],
['1', '8', '3', '1', '1'],
['1', '8', '1', '8', '1'],
['1', '1', '1', '8', '1'],
['8', '8', '1', '1', 'E']
```

输出最小总代价：7
最优路径：`(0,0) → (1,0) → (2,0) → (3,0) → (3,1) → (3,2) → (4,2) → (4,3) → (4,4)`
结果分析：算法选择了一条累计代价最低的路线；此时曼哈顿距离不再是可采纳启发，仅作为估计项引导搜索。

4.3 测试3：Alpha-Beta博弈树

预设博弈树结构，根节点为MAX玩家。
输出最优博弈得分：5
递归与剪枝过程分析:
初始化调用：`alpha_beta(root, α=-∞, β=+∞, is_max=True)`

1. 根节点 MAX，先访问第一个子节点 `min1` （MIN 节点，$\alpha = -\infty, \beta = +\infty$）
   - `min1` 访问第一个子节点 `max1` （MAX）：叶子 3、5 → MAX 取最大值 **5**，返回 5。
   - `min1` 访问第二个子节点 `max2` （MAX）：叶子 6、9 → MAX 取最大值 **9**，返回 9。
   - `min1` 是 MIN 节点，在 5、9 之间取较小值，得到 `min1=5` ；此时根节点的α更新为 5。
2. 根节点 MAX 继续访问第二个子节点 `min2` （MIN 节点，此时传入$\boldsymbol{\alpha = 5, \beta = +\infty}$）
   - `min2` 首先访问第一个子节点 `max2` ，返回值 9。
   - `min2` 作为 MIN，目前候选值为 9，更新$\boldsymbol{\beta = 9}$。
   - 判断条件 $\boldsymbol{\alpha \geq \beta} \Rightarrow 5 \geq 9$不成立，**不触发剪枝**，继续访问 `min2` 下一个子节点 `max3` 。
   - `max3` 是 MAX 节点，叶子 1、2，取最大值返回 2。
   - `min2` (MIN) 在 9、2 之间取更小值得到 `min2=2` 。
3. 回到根 MAX 节点，子节点结果是 `min1=5` ， `min2=2` ；MAX 取两者最大值，最终结果**5**。
   ✨本样例**没有发生剪枝**：因为子节点顺序，没有出现满足$\alpha \geq \beta$的条件。
   拓展思考：如果调换子节点访问顺序，就会触发剪枝，减少叶子节点遍历数量。原始 `Minimax` 算法会遍历全部 6 个叶子节点；本样例 Alpha-Beta 同样遍历全部叶子。 这也体现一个考点：**Alpha-Beta 不一定每次都会剪枝；剪枝效果高度依赖子节点遍历顺序**。

4.4 测试 4：竞争环境下宝藏分配综合题

任务：玩家 A (MAX)、B (MIN) 轮流抢夺宝藏；A-B-A-B 共 4 回合；分别在搜索深度 H=2、H=4 下求解 A 的最优选择。
关键点：A 起点((0,0))，B 起点((6,6))；宝藏 `T1、T2、T3、T4` 。 输出结果：

1. H=2（只预测 A、B 两轮行动）：`{"best_target": "T2", "best_value": -1}`
2. H=4（完整预测全部四轮行动）：`{"best_target": "T3", "best_value": 3}`
   结果分析：
   H=2 时，A 仅能预判自己行动之后 B 的一步反制，只能看到局部局面，选择局部最优 T2，最终收益差-1。
   H=4 可以完整预判全部 4 回合博弈，能够看到后续所有回合对抗，选择全局最优 T3，收益差为 3。

> 现象：**搜索深度不足会带来短视问题，浅层搜索得到局部最优，不等于全局最优**。Alpha-Beta 剪枝在 H=4 搜索中裁剪部分无效分支，相比纯 Minimax 减少访问状态数量，输出决策与 Minimax 完全一致。

5 结果分析与思考

1. A*算法：启发函数直接决定搜索性能。只有满足可采纳条件，A*才保证全局最优解。加权地形中曼哈顿距离失去可采纳性，即便样例输出正确，不能保证所有地图得到最优。
2. Alpha-Beta 剪枝：剪枝只优化搜索效率，不会改变博弈结果。剪枝不是必然触发；子节点访问顺序极大影响剪枝收益，最坏情况等价原始 Minimax。
3. 宝藏争夺综合实验：深度受限对抗搜索存在短视缺陷，深度越大，决策越接近全局最优，但博弈树节点数量指数级膨胀，计算开销快速增大。预计算 A * 距离矩阵，避免递归内部重复路径计算，是工程上重要的优化手段。
4. 边界鲁棒性：迷宫完全无路返回空列表；加权地图无解返回-1；宝藏取完或者达到深度限制博弈立刻终止，程序对边界场景具备处理能力。
5. 算法局限：本实验仅支持四方向移动；对抗搜索随着可选动作增多，状态爆炸严重，实际项目中常使用启发评估函数、剪枝、状态缓存来降低开销。

6 实验总结

本次实验完成四项任务：无权迷宫 A*寻路、加权地形 A*最小代价搜索、基础博弈树 Alpha-Beta 剪枝、A*结合对抗搜索的宝藏争夺综合问题。 通过实验理解评估函数 (f(n)=g(n)+h(n))的含义，分清可采纳启发函数的约束条件，掌握 A*得到最优解的前提。掌握 MAX-MIN 交替的博弈递归思想，理解 Alpha-Beta 剪枝生效条件，明白遍历顺序对剪枝的影响。综合题体会深度受限对抗搜索的短视问题，理解预测深度对博弈决策质量的影响。全部测试用例运行结果均符合预期，完成本次实验全部要求。